

Job-Ready AI

PROJECT ARCHITECTURE & IMPLEMENTATION REPORT

1. Introduction

Job-Ready AI is an intelligent, full-stack application designed to optimize the job application process for candidates. In today's highly competitive job market, candidates face the significant challenge of tailoring their resumes to bypass Applicant Tracking Systems (ATS). Generic resumes often score poorly, leading to premature rejections despite a candidate's actual qualifications.

This project addresses that gap by providing a dual-workflow platform. It enables users to either build a perfectly formatted resume from scratch using a guided interface, or upload an existing resume and have it dynamically tailored to a specific job description. By leveraging advanced Large Language Models (LLMs) and semantic vector embeddings, Job-Ready AI automates the extraction of job requirements, evaluates skill gaps, calculates ATS compatibility, and generates highly targeted application materials including optimized resumes, tailored cover letters, and personalized cold emails.

2. System Architecture

The application is built using a modern Python data science stack, structured to separate the user interface, workflow management, text processing, and AI generation logic. The architecture relies on external APIs for heavy language processing while handling vector embeddings locally for speed and efficiency.

Frontend & UI Layer

Streamlit: The core user interface framework (`main.py`). It manages application state, multi-step forms, file uploads, and renders dynamic results including progress bars, metric displays for ATS scores, and downloadable artifacts.

Application Logic & State	Workflow Manager: Handles the routing between the "Create New" and "Tailor Existing" paths, manages temporary file ingestion, and provides UI utilities. Resume Form: A stateful, multi-step interactive form component managing personal data, education, experience, and skill inputs.
Language Model (LLM) Integration	LangChain & Groq: The application utilizes the <code>ChatGroq</code> interface via LangChain. It connects to powerful open-source models (such as <code>llama-3.3-70b-versatile</code>) for complex tasks: parsing unstructured job descriptions into JSON, generating cover letters, rewriting resume bullet points for ATS optimization, and analyzing skill gaps.
Semantic Search & Embeddings	Sentence-Transformers: Employs the local <code>all-MiniLM-L6-v2</code> model to convert both resume sentences and required job skills into dense vector embeddings. NumPy: Used to compute the normalized cosine similarity between the resume text vectors and skill vectors, allowing the system to identify implicitly mentioned skills rather than relying strictly on exact keyword matching.
Document Processing Engine	Ingestion: <code>PyPDF2</code> and <code>python-docx</code> for extracting raw text from user-uploaded resumes. <code>BeautifulSoup</code> and <code>WebBaseLoader</code> for scraping job descriptions directly from URLs. Generation: <code>Jinja2</code> templates compile form data into structured HTML. <code>pdfkit</code> converts the HTML into professional PDF documents, while <code>python-docx</code> is used to generate downloadable, ATS-friendly Word documents of the tailored output.

3. Functionality Implemented

The application is divided into two primary user workflows, each handling a specific phase of career preparation.

3.1. Workflow A: Create New Resume

Designed for users who need to build a structured, professional resume from scratch. This workflow utilizes the `ResumeForm` and `ResumeGenerator` modules.

- **Interactive Multi-Step Form:** Guides the user through a 5-step process collecting Personal Information, Education history, Work Experience, Technical/Soft Skills, and Certifications.
- **State Management:** Ensures data persists across UI interactions, allowing users to go back and edit previous steps before finalizing.
- **Dynamic Rendering & Export:** Upon completion, the system injects the collected data into a Jinja2 HTML template. Users are provided with an interactive live preview and the ability to download the final result as a fully formatted PDF or HTML file.

3.2. Workflow B: Tailor Existing Resume

The core AI-driven feature of the application, designed to optimize an existing resume against a target job posting.

1. Intelligent Ingestion & Parsing

Users upload their current resume (PDF/DOCX) and provide a job posting (via direct text paste or URL). The LLM processes the job posting and extracts a structured JSON schema containing the required skills, necessary experience, core responsibilities, and company details.

2. Advanced Skill Matching

The system performs a two-pass matching algorithm. First, it uses an extensive dictionary of tech variations (e.g., matching "React.js" to "React") for exact keyword hits. Second, it utilizes semantic vector similarity to detect skills that are described contextually in the resume but not explicitly named, categorizing the results into "Matched Skills" and "Missing Skills".

3. ATS Optimization & Rewriting

The LLM is prompted to act as an ATS optimization specialist. It rewrites the user's resume to naturally incorporate missing keywords and align experiences with the target role. Strict system prompts ensure the model does not hallucinate skills the user does not possess.

4. ATS Score Calculation

The system calculates a comprehensive 0-100 score for the resume both *before* and *after* tailoring. It evaluates Keyword Matches (30 pts), Experience Alignment (25 pts), Skills Match (25 pts), Education Relevance (10 pts), and Formatting (10 pts), providing the user with a detailed breakdown of improvements.

5. Outreach Material Generation

Finally, the system utilizes the newly optimized resume and job context to generate two highly targeted outreach artifacts: a professional Cover Letter formatted with standard business conventions, and a concise, high-impact Cold Email designed to highlight top qualifications to a hiring manager.

End of Report.